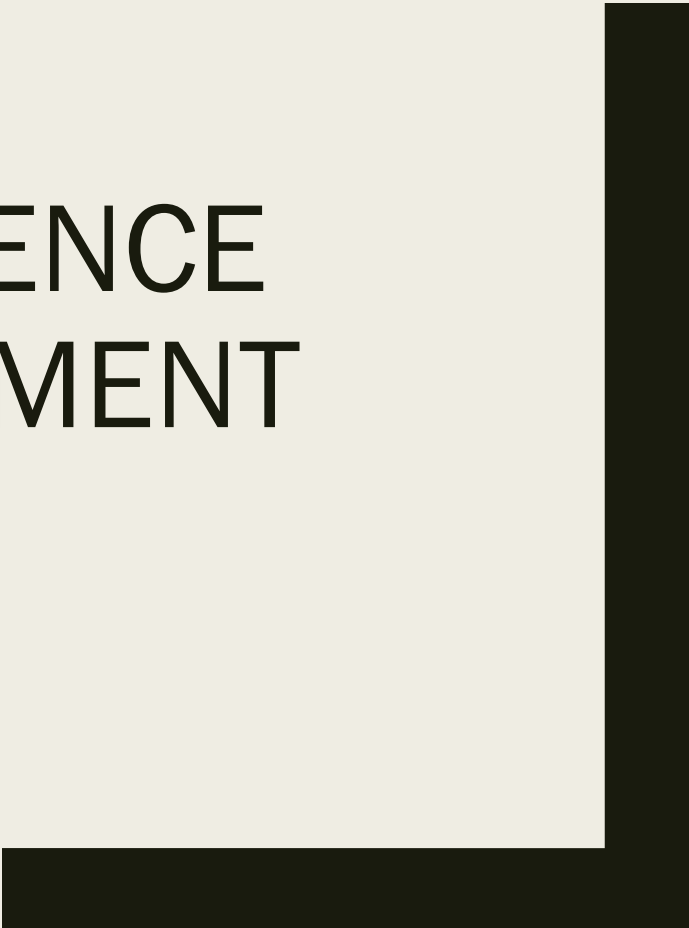




ARTIFICIAL INTELLIGENCE SOFTWARE DEVELOPMENT

CST8510 Week 11
Dr. Hari M Koduvely



Agenda for Today

- ❑ Theory: 5:30PM – 7:30PM
 - Prompt Optimization
 - Reflection Methods
 - Advanced RAG Techniques
- ❑ Lab: 7:30PM – 9:30PM
 - Standup Meetings



Prompt Optimization

❑ Uncertainty Thresholding with Self-Consistency

“Opt-out” confidence thresholding

- **Implicit approach:** LLM self-confidence evaluation assigns human support, general LLM, or a user-friendly “I don’t know” message as a prompt response “out”
- **Explicit approach:** Rule-based threshold through self-consistency
 - **Uncertainty:** Proportion of answers that are unique
 - **Token likelihood:** Likelihood of an LLM output (cumulative token probability)

SYSTEM

I want you to act as a virtual air travel agent with expertise in customer support that receives, as input, customer requests for information and assistance pertaining to the end-to-end airport experience. Please categorize each request into one of the following categories:

- Flight Bookings: Requests for flight bookings, cancellations, modifications, or upgrades
- Flight Information: Requests for flight details, live schedule information, boarding times, safety protocol, and other flight procedures
- Flight Resources: Requests to track or identify baggage, locate transportation services post-flight, or find available curbside and parking locations and resources pre-flight

For example, a user request of “I’m noticing some heavy rain in my area. Should I expect any delays for my flight that departs in 2 hours?” would receive a Flight Information label.

Please only provide the request label in your response. If you are unsure of the proper label to assign based on the context above, please label the request with Human Support so the customer can get directed to a live agent.

• Persona introduction

• Capability articulation

• Definition of scope

• Placeholder for multi-shot

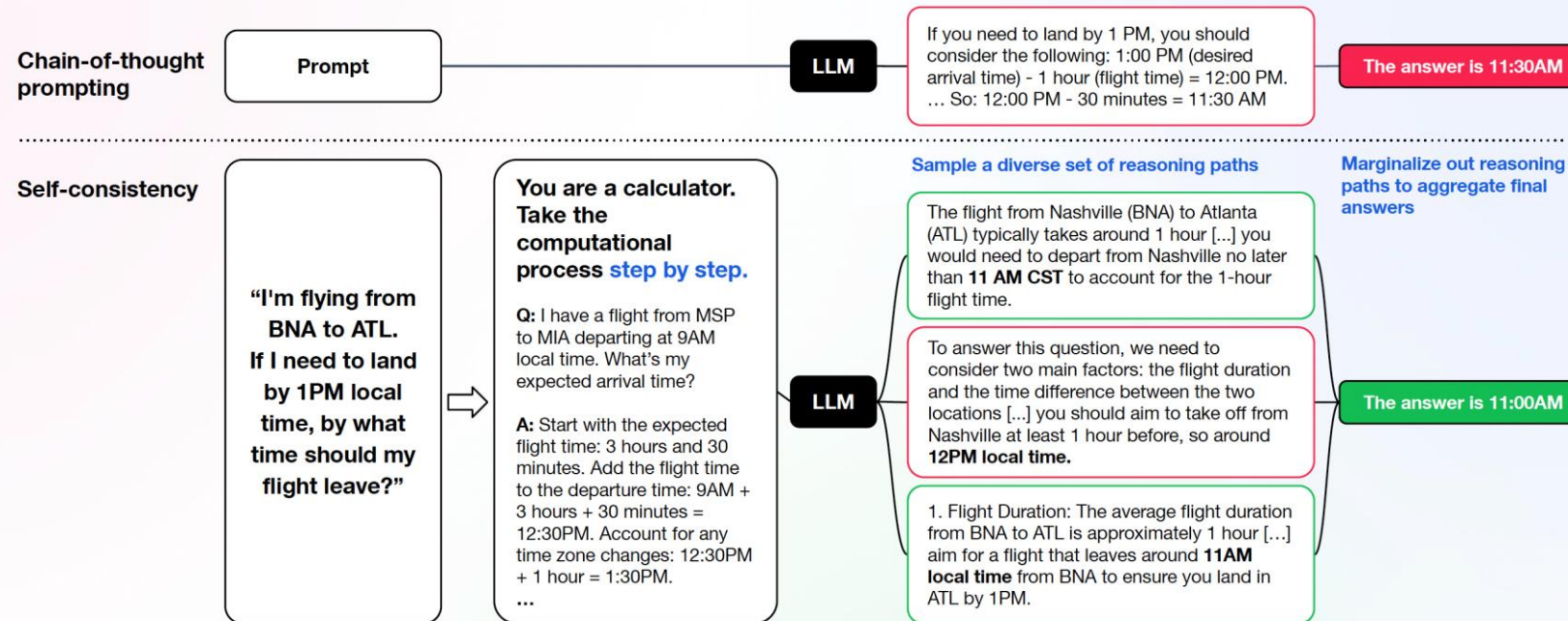
• Output specification with “opt-out”

Prompt Optimization

❑ Uncertainty Thresholding with Self-Consistency

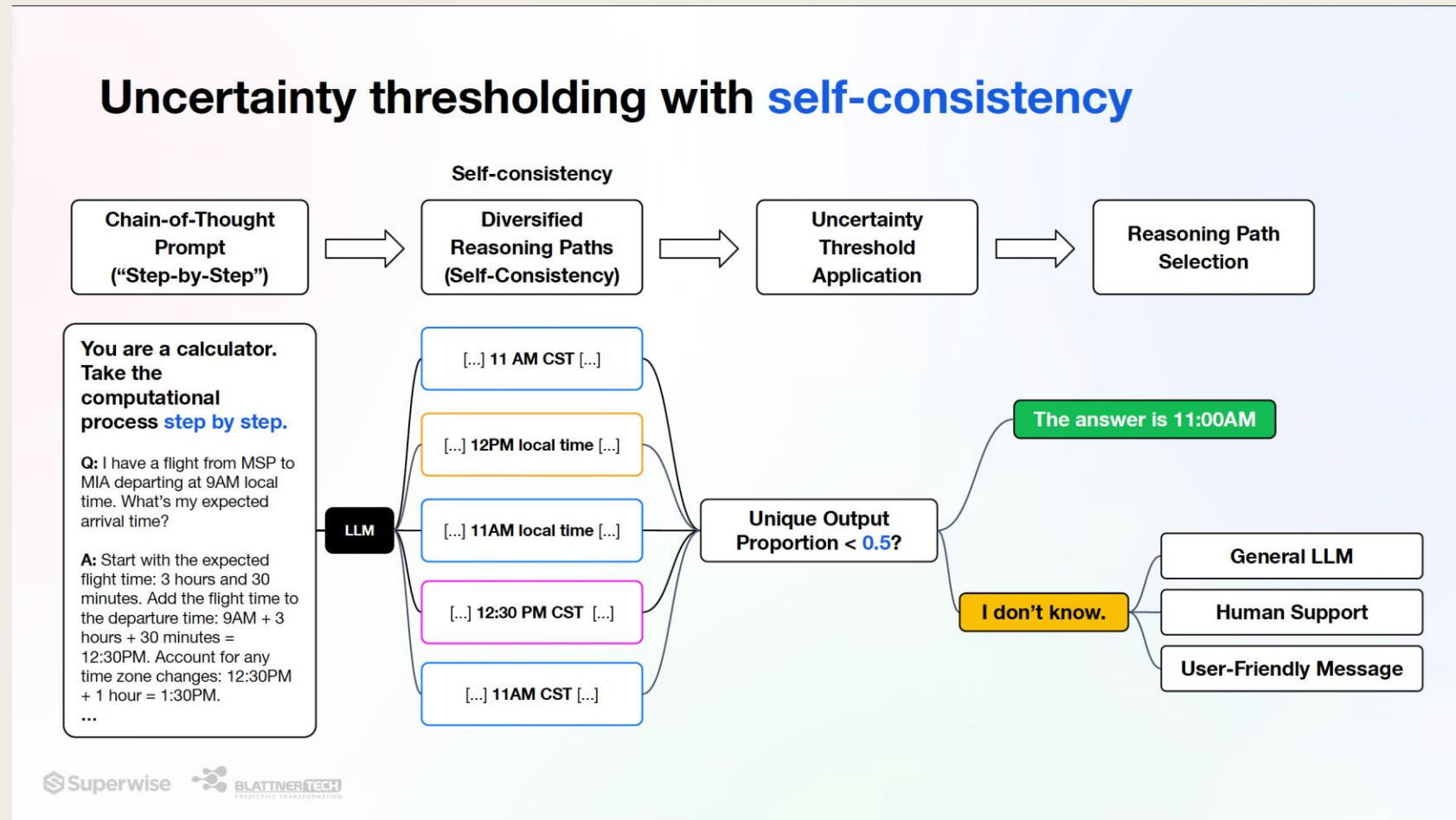
Optimizing prompt accuracy: **Self-consistency**

Improve the consistency of an existing LLM system without altering its architecture



Prompt Optimization

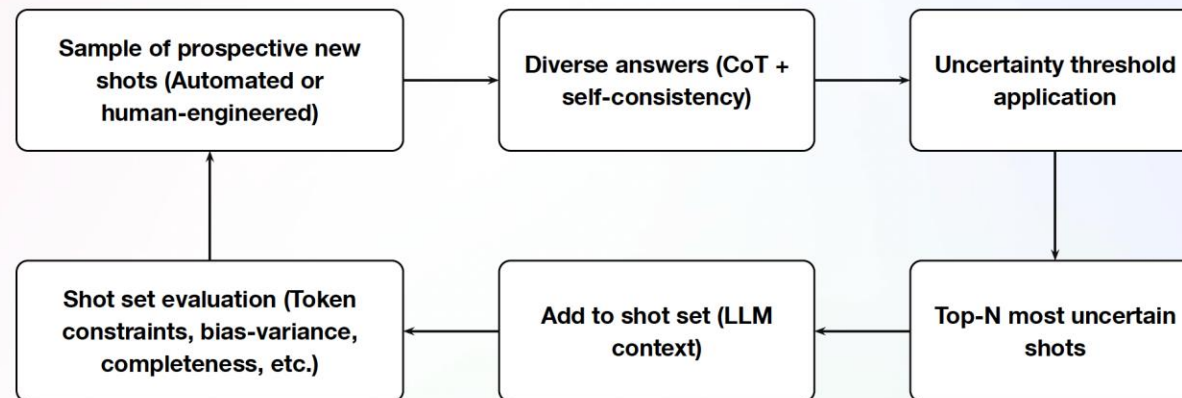
❑ Uncertainty Thresholding with Self-Consistency



Prompt Optimization

❑ Uncertainty Thresholding with Self-Consistency

Active prompting with **uncertainty thresholding**



OPRO – Optmization by PROmpting

- ❑ OPRO is a simple yet effective technique that employs LLMs as optimizers.
- ❑ The optimization task is described in natural language.
- ❑ LLM generates new solutions based on the problem description and previously found solutions.
- ❑ This iterative process converges to an optimal solution.

OPRO – How it Works?

- ❑ **Meta-Prompt:** This is the input to the LLM, containing the problem description, solution constraints, and previously evaluated solutions with their corresponding scores.
- ❑ **LLM as Optimizer:** The LLM generates new candidate solutions based on the information in the meta-prompt.
- ❑ **Objective Function Evaluator:** This component evaluates the new solutions and provides feedback to the LLM, which is then used to refine the meta-prompt for the next optimization step.

OPRO – How it Works?

- ❑ **Meta-Prompt:** This is the input to the LLM, containing the problem description, solution constraints, and previously evaluated solutions with their corresponding scores.
- ❑ **LLM as Optimizer:** The LLM generates new candidate solutions based on the information in the meta-prompt.
- ❑ **Objective Function Evaluator:** This component evaluates the new solutions and provides feedback to the LLM, which is then used to refine the meta-prompt for the next optimization step.

OPRO – Illustration

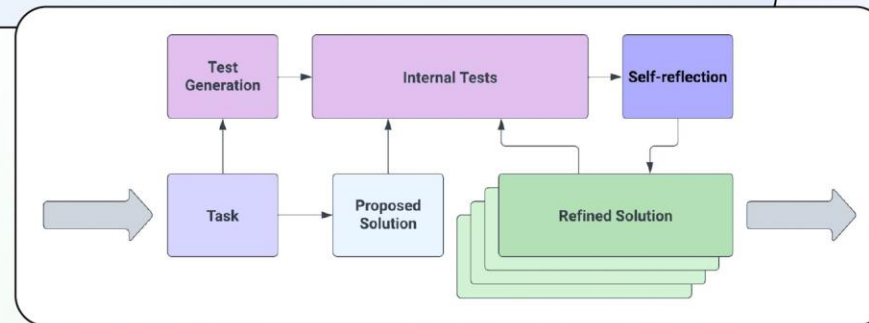
GSM8K Dataset – Grade School Math Problems Dataset

Take a deep breath and work on this problem step-by-step.	80.2
Break this down.	79.9
A little bit of arithmetic and a logical approach will help us quickly arrive at the solution to this problem.	78.5
Let's combine our numerical command and clear thinking to quickly and accurately decipher the answer.	74.5

Self-reflection

Self-evaluation: Reflexion

- For use cases with a ground truth that is out of reach: a feedback loop that iteratively approaches high-confidence outputs
 - a. Correction of errors, especially for multi-step reasoning or code generation tasks
 - b. Improvement of output quality, especially for requirement satisfaction
 - c. Boost in contextual understanding through evaluation of interpretations
- Two core approaches:
 - a. Feedback-Driven Reflexion: Iterative changes via self-assessment are triggered by negative user feedback (👍 / 👎 feature for chat-style interfaces, etc.)
 - b. Implicit Reflexion: Self-assessment is applied on all requests before initial front-end delivery



Self-reflection

Feedback-driven reflexion

My flight has been delayed. Where can I go for more information on my flight's status?

I'm sorry to hear that! For more details on your flight, please visit our Flight Status page.

Response was labeled poor (General)

Response was labeled incomplete (Specific)

**Feedback-Driven
Self-Reflection Layer**

Please modify the **relevance** of your previous response to the customer's request for up-to-date flight status information. Offer a more **specific and helpful** solution.

Please modify the [**completeness**, accuracy, helpfulness, specificity] of your previous response to the customer's request for up-to-date flight status information. Offer a more [**complete**, accurate, helpful, specific] solutions.

I'm sorry to hear that! For more information on your delayed flight, please consider consulting one of the following resources:

1. Airport Website: Please visit our Flight Status page for real-time information about flight status, delays, and any changes to the schedule.
2. Airline Kiosk: Ask an airport representative for updated flight details before entering a security checkpoint
3. Airport Information Screens: Your airport will display real-time flight information on departure and arrival screens located throughout the terminals.

Self-reflection

Implicit reflexion

My flight has been delayed. Where can I go for more information on my flight's status?

I'm sorry to hear that! For more details on your flight, please visit our Flight Status page.

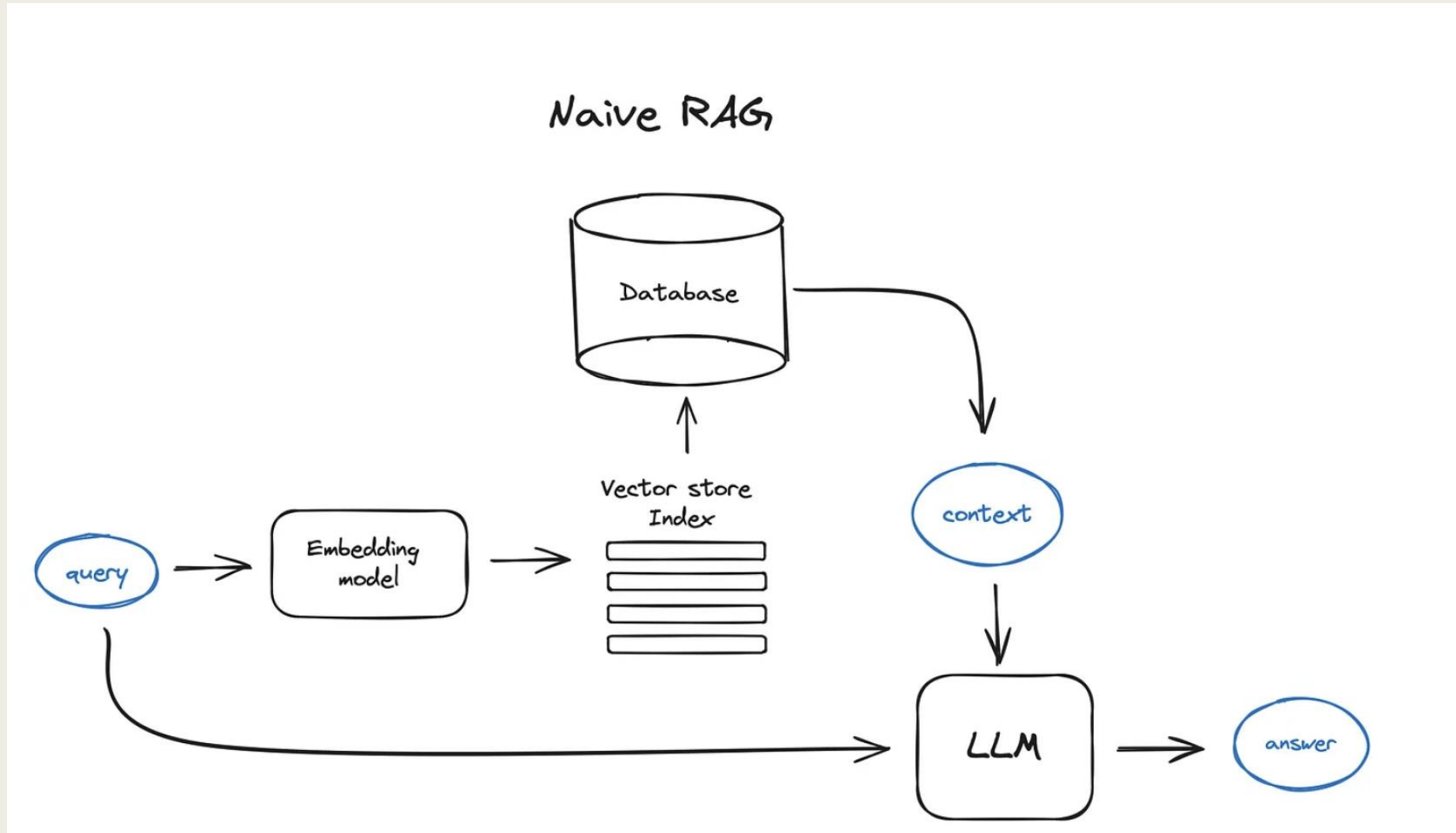
Hidden Self-Reflection
Layer

Evaluate the **relevance** of your previous response to the customer's issue of intermittent internet connection. Offer a more specific and helpful solution **if necessary**.

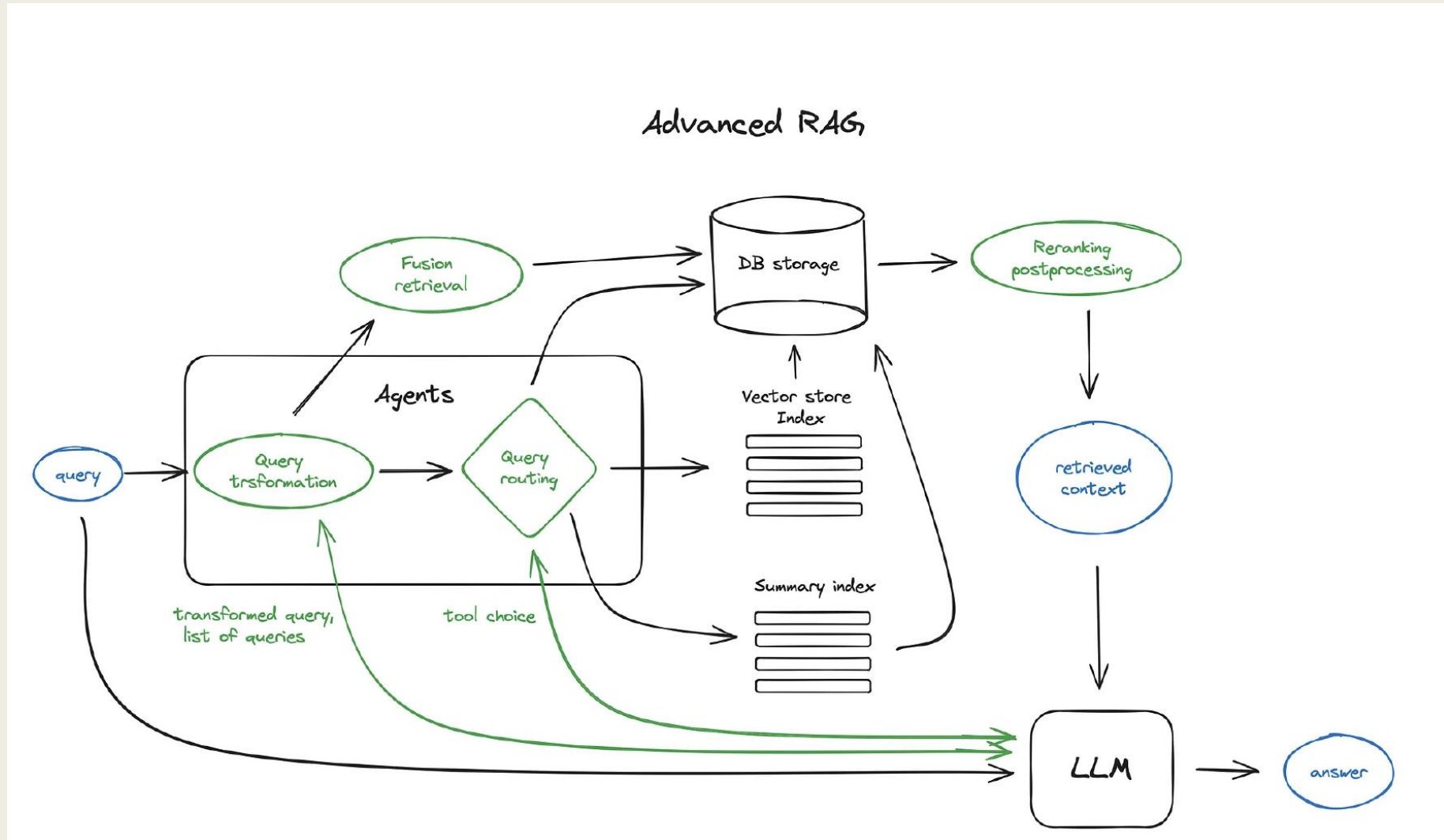
I'm sorry to hear that! For more information on your delayed flight, please consider consulting one of the following resources:

1. Airport Website: Please visit our Flight Status page for real-time information about flight status, delays, and any changes to the schedule.
2. Airline Kiosk: Ask an airport representative for updated flight details before entering a security checkpoint
3. Airport Information Screens: Your airport will display real-time flight information on departure and arrival screens located throughout the terminals.

Advanced RAG



Advanced RAG



Advanced RAG – Improvement in Chunking

Unstructured chunking

- **Character splitting (size-based):** Chunking text into equally sized documents based on character length, which may result in:
 - Multiple topics in one chunk
 - One topic in multiple chunks
- **Unstructured chunking:** Content-aware identification of text objects via syntactical rules
- **Benefit:** Assigning one chunk to each text object produces semantically consistent documents of varying lengths
- **Drawback:** Syntactical similarity + proximity $\neq$ semantic similarity in every case

Character Splitting

3.1 Open-domain Question Answering

Open-domain question answering (QA) is an important real-world application and common testbed for knowledge-intensive tasks [20]. We treat questions and answers as input-output text pairs (x, y) and train RAG by directly minimizing the negative log-likelihood of answers. We compare RAG to the popular extractive QA paradigm [5, 7, 31, 26], where answers are extracted spans from retrieved documents, relying primarily on non-parametric knowledge. We also compare to “Closed-Book QA” approaches [52], which, like RAG, generate answers, but which do not exploit retrieval, instead relying purely on parametric knowledge. We consider four popular open-domain QA datasets: Natural Questions (NQ) [29], TriviaQA (TQA) [24], WebQuestions (WQ) [3] and CuratedTrec (CT) [2]. As CT and WQ are small, we follow DPR [26] by initializing CT and WQ models with our NQ RAG model. We use the same train/dev/test splits as prior work [31, 26] and report Exact Match (EM) scores. For TQA, to compare with T5 [52], we also evaluate on the TQA Wiki test set.

3.2 Abstractive Question Answering

RAG models can go beyond simple extractive QA and answer questions with free-form, abstractive text generation. To test RAG’s natural language generation (NLG) in a knowledge-intensive setting, we use the MSMARCO NLG task v2.1 [43]. The task consists of questions, ten gold passages retrieved from a search engine for each question, and a full sentence answer annotated from the retrieved passages. We do not use the supplied passages, only the questions and answers, to treat

Unstructured Chunking

3.1 Open-domain Question Answering

Open-domain question answering (QA) is an important real-world application and common testbed for knowledge-intensive tasks [20]. We treat questions and answers as input-output text pairs (x, y) and train RAG by directly minimizing the negative log-likelihood of answers. We compare RAG to the popular extractive QA paradigm [5, 7, 31, 26], where answers are extracted spans from retrieved documents, relying primarily on non-parametric knowledge. We also compare to “Closed-Book QA” approaches [52], which, like RAG, generate answers, but which do not exploit retrieval, instead relying purely on parametric knowledge. We consider four popular open-domain QA datasets: Natural Questions (NQ) [29], TriviaQA (TQA) [24], WebQuestions (WQ) [3] and CuratedTrec (CT) [2]. As CT and WQ are small, we follow DPR [26] by initializing CT and WQ models with our NQ RAG model. We use the same train/dev/test splits as prior work [31, 26] and report Exact Match (EM) scores. For TQA, to compare with T5 [52], we also evaluate on the TQA Wiki test set.

3.2 Abstractive Question Answering

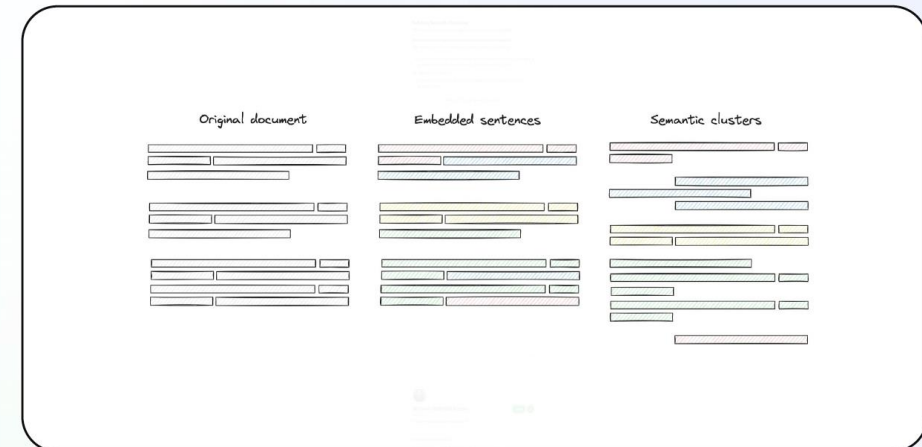
RAG models can go beyond simple extractive QA and answer questions with free-form, abstractive text generation. To test RAG’s natural language generation (NLG) in a knowledge-intensive setting, we use the MSMARCO NLG task v2.1 [43]. The task consists of questions, ten gold passages retrieved from a search engine for each question, and a full sentence answer annotated from the retrieved passages. We do not use the supplied passages, only the questions and answers, to treat

<https://medium.com/unstructured-io/rag-isnt-so-easy-why-llm-apps-are-challenging-and-how-unstructured-can-help-8daaf859c615>

Advanced RAG – Improvement in Chunking

Semantic chunking (Contextual compression)

- Leverages sentence-level semantic similarity methods in order to order chunk-level content more meaningfully
 - Split documents into sentences
 - Sentence embeddings
 - Cluster semantically similar sentences and re-indexing
- Built-in **contextual compression** minimizes unneeded noise sent to the LLM
- Benefit: Semantically tight chunks ensure minimal loss
- Drawback: Contextual richness of the original corpus structure is lost

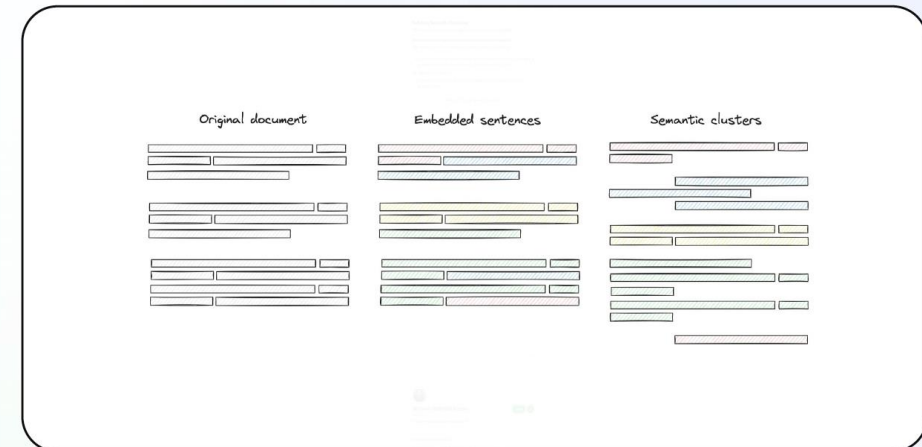


<https://medium.com/@hadiadzouni/text-splitting-chunking-for-rag-applications-7ccbb6dcc9f9>

Advanced RAG – Improvement in Chunking

Semantic chunking (Contextual compression)

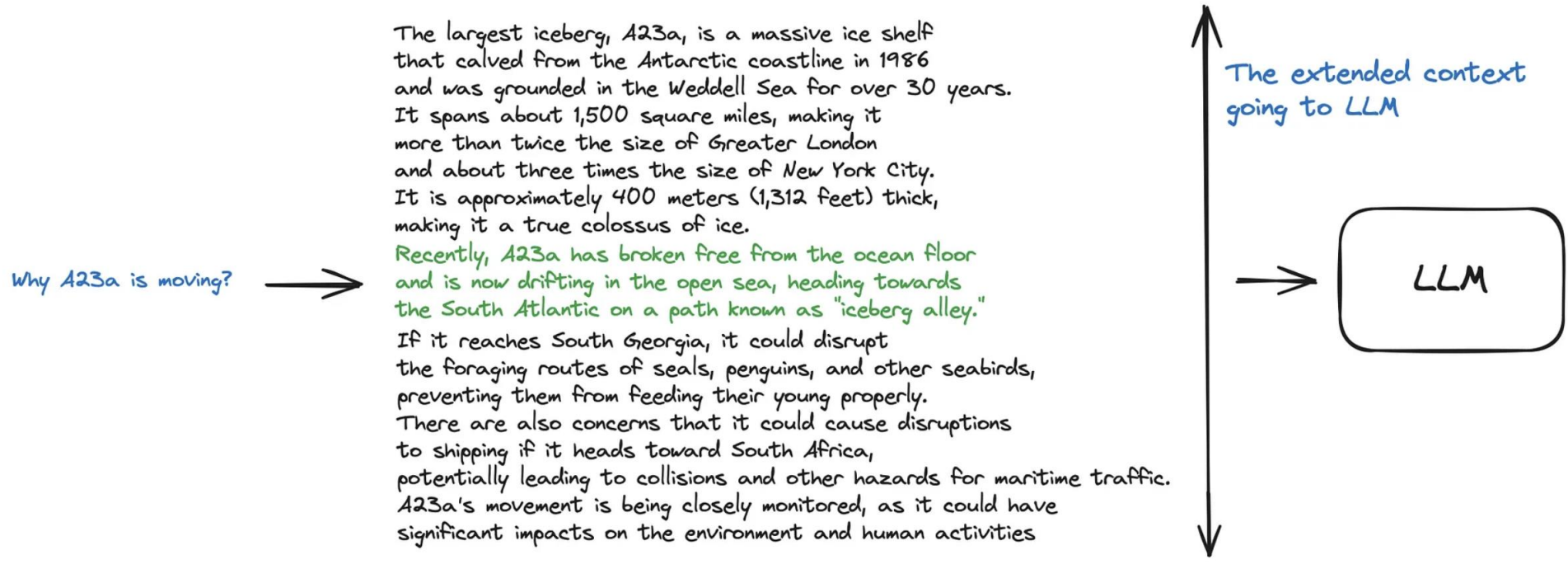
- Leverages sentence-level semantic similarity methods in order to order chunk-level content more meaningfully
 - Split documents into sentences
 - Sentence embeddings
 - Cluster semantically similar sentences and re-indexing
- Built-in **contextual compression** minimizes unneeded noise sent to the LLM
- Benefit: Semantically tight chunks ensure minimal loss
- Drawback: Contextual richness of the original corpus structure is lost



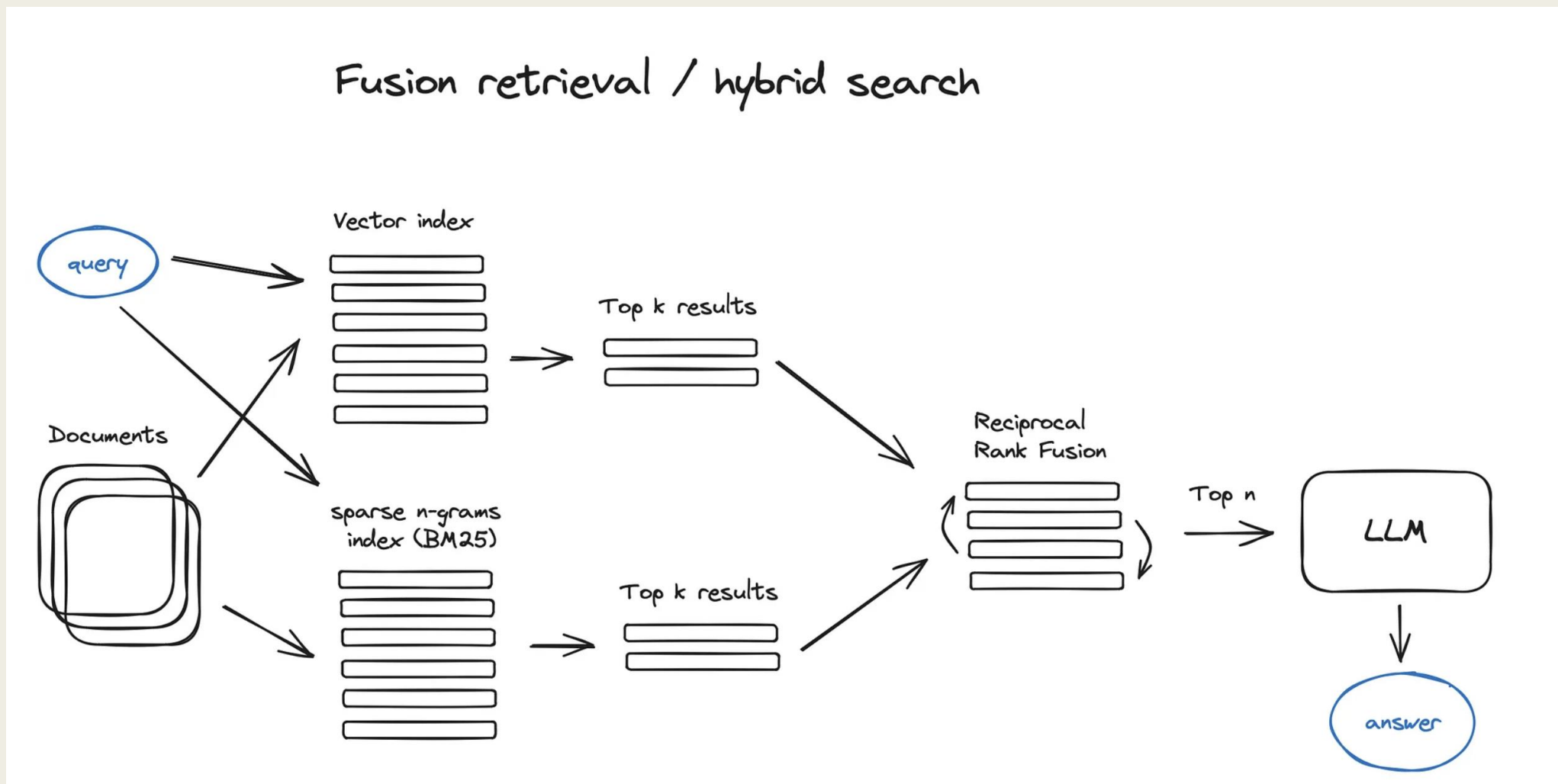
<https://medium.com/@hadiazouni/text-splitting-chunking-for-rag-applications-7ccbb6dcc9f9>

Advanced RAG – Extending Context Window

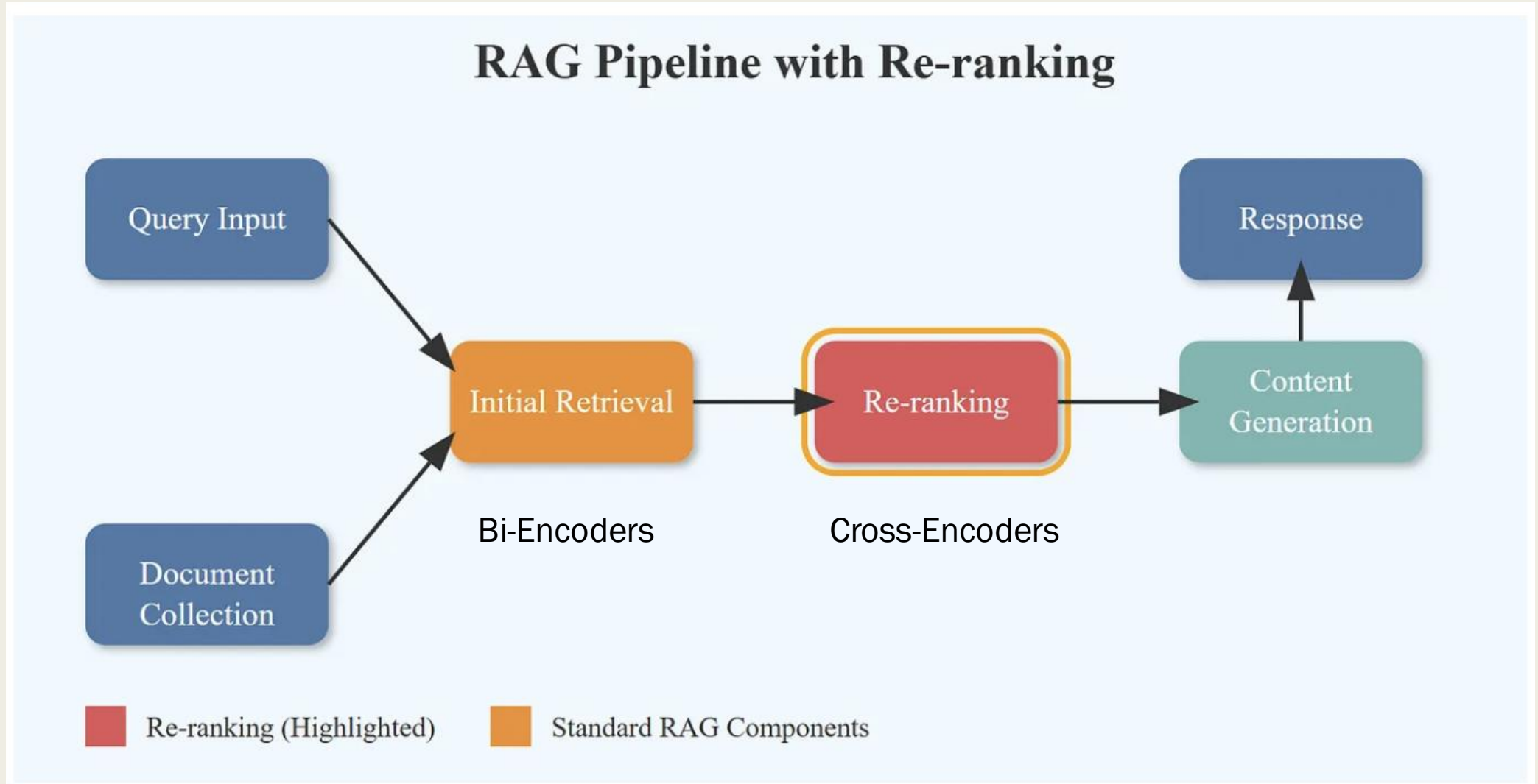
Sentence Window Retrieval



Advanced RAG – Hybrid Search



Advanced RAG – Reranking



Advanced RAG – Reranking

Relevance Scoring Models:

- The system assigns relevance scores to each document
- Use models like BERT-based rerankers or cross-encoders
- Considers both the query and retrieved result together

Traditional Ranking Algorithms

- BM25 (considers term frequency and document length)

Advanced RAG – Reranking

- Can improve quality of retrieval significantly
- Prone to increasing the latency
- Additional computational cost also can incur

Advanced RAG – Fine-Tuning Embedding Models

- Embedding models are trained using standard dataset corpus
- They may not capture the deep semantic relationships in a document
- Fine-tune embedding models with your domain data helps
- More cost effective than fine-tuning LLM

Summary of Today's Learning

- ❑ Different prompt optimization methods.
- ❑ Reflection methods – self and feedback driven.
- ❑ Improving RAG through:
 - Better Chunking
 - Reranking
 - Fine-tuning embedding models

Thank You